

数据库系统实验课程设计报告

郑皓	学号：23336328	班级：9
姚书璟	学号：23336386	班级：8
罗斯	学号：xxx	班级：9

提交时间：_____ 年 _____ 月 _____ 日

课程设计题目

基于 MiniOB 的数据库内核设计

1 开发环境

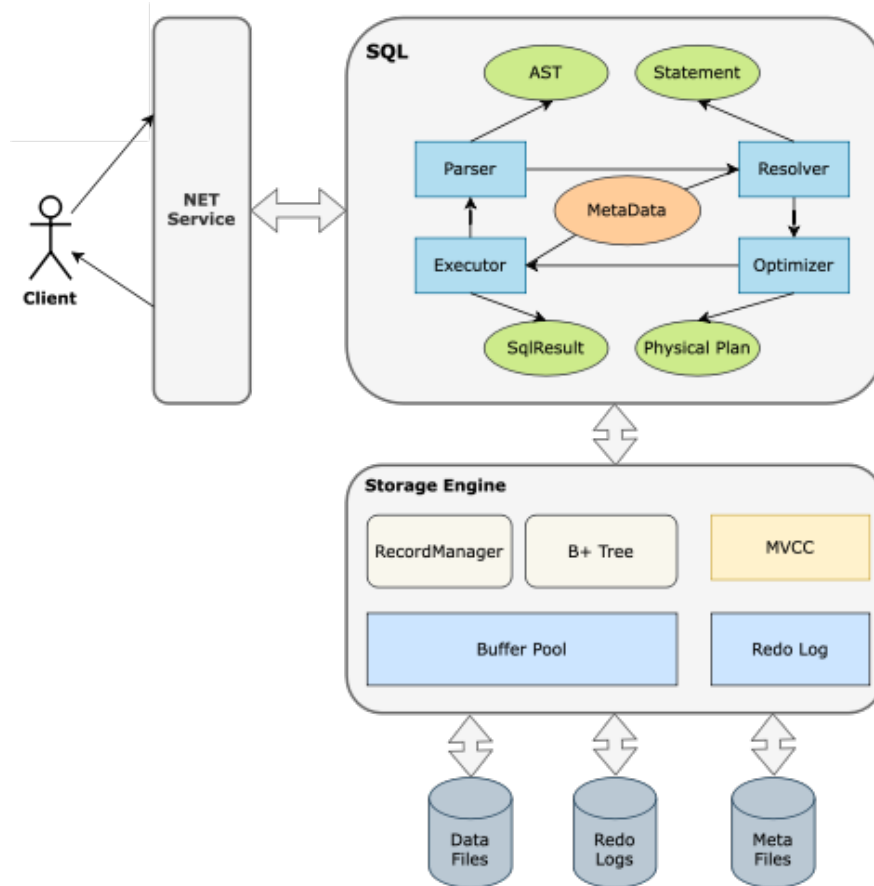
- 开发环境: Windows 11 + WSL
- 开发工具: Docker, Visual Studio Code
- MiniOB: OceanBase 团队基于华中科技大学数据库课程原型开发的入门学习项目

环境配置

- 从 github 的 competition-2025 分支下载源码。
- 使用 `docker run -d --name minioob --privileged -v path_to_src:/root/minioob oceanbase/minioob` 安装 docker 开发环境并绑定本地源码目录(路径为 `path_to_src`)。
- 在使用 vscode 附加到容器后, 使用 `CTRL + SHIFT + P`, 点击 Task: Run Task, 然后点击里面已经设置好的 `cmake` 和 `make` 指令并选中“继续而不扫描任务输出”即可对源码进行编译。
- 编译成功后, 在 Task: Run Task 中找到 Run observer 指令即可运行 minioob 数据库服务器, 再使用 Run obclient 指令即可运行客户端进行交互。

2 MiniOB 系统架构说明

MiniOB 整体架构



- 网络模块 (NET Service)：负责与客户端交互
- SQL 解析 (Parser)：将 SQL 语句解析成语法树
- 语义解析 (Resolver)：语法树转内部数据结构
- 查询优化 (Optimizer)：调整/重写语法树
- 计划执行 (Executor)：执行并生成结果
- 存储引擎 (Storage Engine)：数据存储和检索
- 事务管理 (MVCC)：管理事务提交、回滚
- 日志管理 (Redo Log)：记录操作日志
- B+ Tree：索引存储结构

注： 如上为 MiniOB 整体架构图，展示了各个模块及其交互关系。我们将在模块阐释与实现分析中**重点关注**与存储引擎、B+ 树和文件组织等相关底层模块，淡化对于语法解析器 Parser、执行器 Executor 等上层模块的讨论。

表文件管理

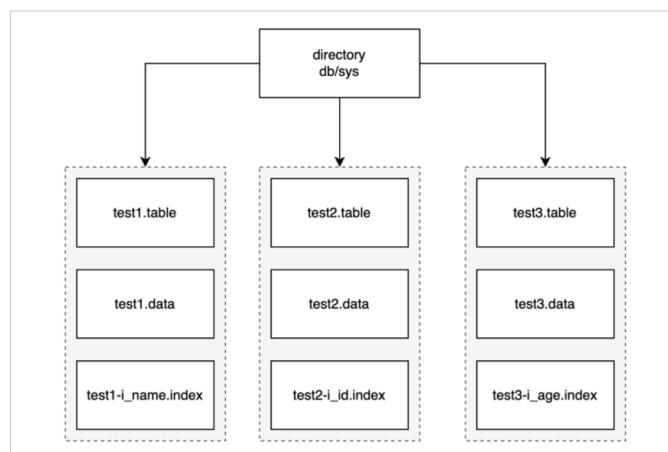


图 1: 表文件组织方式图

- test1.table: 元数据文件，这里面存放了一些元数据。如：表名、数据的索引、字段类型、类型长度等。
- test1.data: 数据文件，真正记录存放的文件。
- test1-i_name.index: 索引文件，索引文件有很多个，这里只展示一个示例。

缓冲区结构

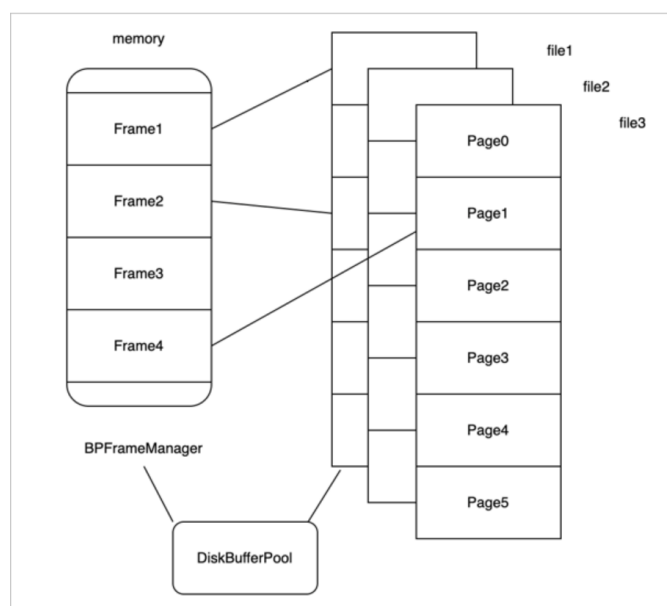


图 2: 缓冲区结构示意图

概述 MiniOB 的缓冲区采用典型的 Buffer Pool 机制。内存被划分为若干固定大小的页帧（frame），磁盘文件按固定大小的页（page）组织，每个 page 在内存中至多对应一个 frame，二者大小一致，数据交换以页为单位进行。

在实现上，每个物理文件对应一个 DiskBufferPool 对象，而所有 DiskBufferPool 共享统一的页帧管理组件 BPFrameManager，由其负责 frame 的分配、回收与淘汰。

当访问某个磁盘页面时，系统首先在 BPFrameManager 中查找空闲 frame；若存在，则将该 frame 与目标 page 绑定，并将页面数据读入内存。若内存中不存在空闲 frame，则需要从已占用的 frame 中选择一个进行淘汰，将其解除绑定后，重新关联到新页面并加载数据。

值得注意的是，MiniOB 采用 LRU（Least Recently Used）算法作为页面替换策略。

文件结构

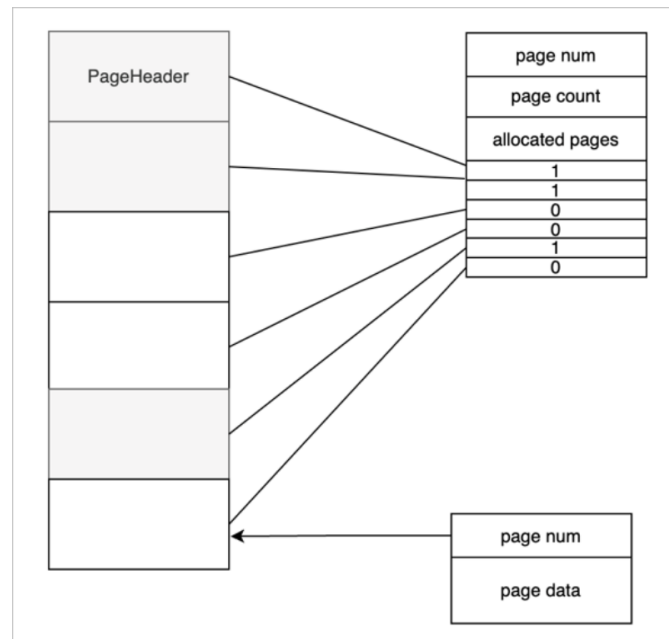


图 3: 文件结构图

- 文件上的第一页称为页头或文件头。文件头是一个特殊的页面，这个页面上会存放一个页号，这个页号肯定都是零号页，即 page num 是 0。
- page count 表示当前的文件一共有多少个页面。
- allocated pages 表示已经分配了多少个页面。如图所示标灰的是已经分配的三个页面。
- Bitmap 表示每一个 bit 位当前对应的页面的分配状态，1 已分配页面，0 空闲页面。

记录管理

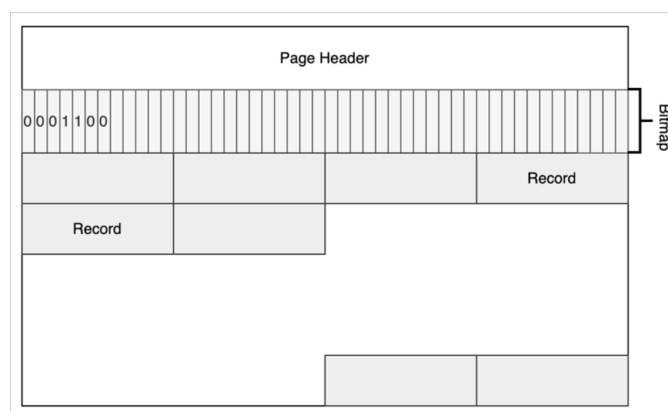


图 4: 记录管理结构图

简述 Record Manager 是在 Buffer Pool 的基础上实现的，比如 page0 是 Buffer Pool 里面使用的元数据，Record Manager 利用了其他的一些页面。每个页面有一个头信息 Page Header，一个 Bitmap，Bitmap 为 0 表示最近的记录是不是已经有有效数据；1 表示有有效数据。Page Header 中记录了当前页面一共有多少记录、最多可以容纳多少记录、每个记录的实际长度与对齐后的长度等信息。

B+ 树

索引文件 每一个索引文件都存储着一棵 B+ 树，其第一个 page 为树的根节点。

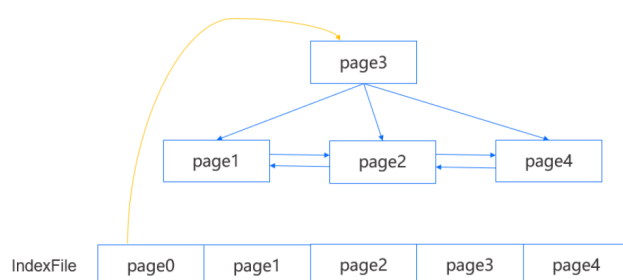


图 5: 索引文件图

叶子节点 B+ 树中的每个结点都存放在一个固定大小的 page 中。每个 page 在文件中由 page_num 唯一标识。结点页都包含一个公共头部 (IndexNode)，其中记录了结点是否为叶结点 (is_leaf)、当前结点中键的数量 (key_num) 以及父结点的页号 (parent，为 -1 表示无父结点)。

叶子结点在公共头部之外，还维护了左右兄弟结点的页号 (prev_brother 和 next_brother)，用于支持顺序遍历和范围查询。页中剩余空间按顺序存放键值数据：每个键由索引列的值和对应记录的 RID 组成，Value 同样为 RID。因此，所有索引键及其对应的数据定位信息均存放在叶子结点中，符合 B+ 树“数据集中在叶子结点”的设计特点。

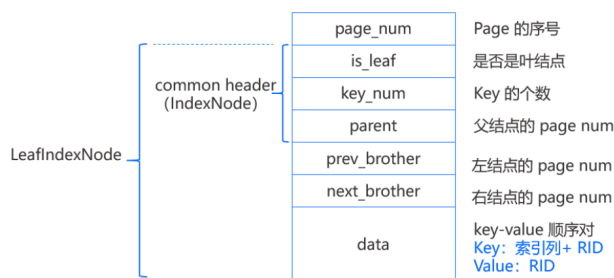


图 6: 叶子节点图

内部节点 内部结点和叶结点有两点不同，一个是没有左右结点的 page num；另一个是所存放的值是 page num，也就是标识了子结点的 page 位置。如下图所示，键值对在内部结点是这样表示的，第一个键值对中的键是一个无效数据，真正用于比较的只有 k_1 和 k_2 。

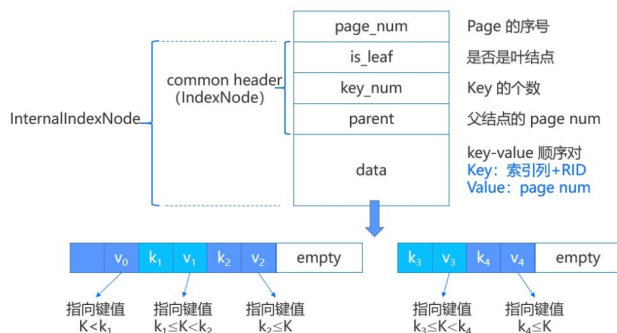


图 7: 内部节点图

3 功能设计与代码实现

3.1 NULL 功能实现

3.1.1 概述

在数据库中，NULL 用于表示缺失或未知的值。MiniOB 对 NULL 的支持贯穿查询解析、执行和存储。在本节中，我们重点关注存储引擎等偏底层的位置如何处理 NULL 值的相关实现。如下是我开发阶段绘制的实现流程思维导图，包含具体实现中的部分核心内容，可供参考：

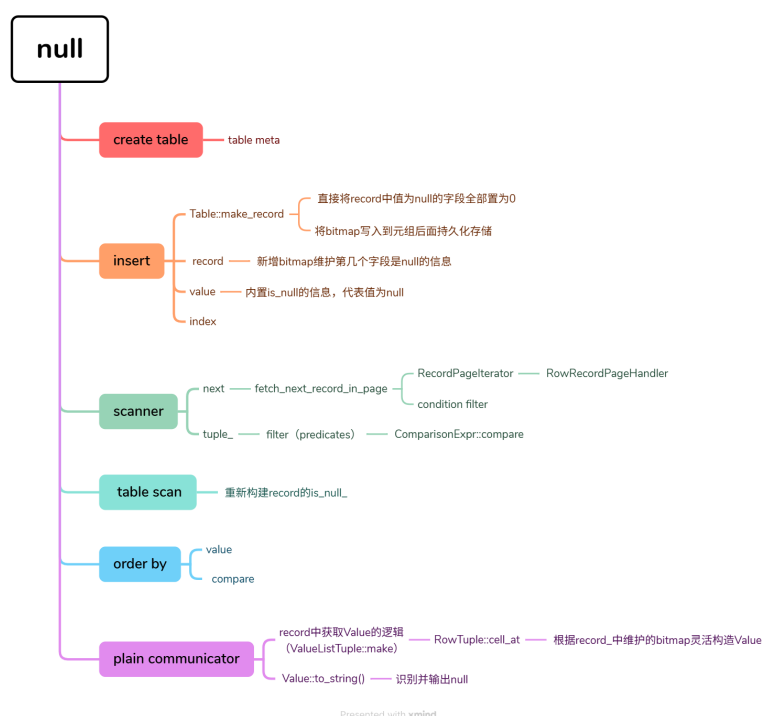


图 8: NULL 实现流程图

3.1.2 语法与词法解析

- 词法解析 (Lexer): 将 SQL 输入分解为 token，包括关键字 NULL、表名、列名等。

- 语法解析 (**Parser**)：构建抽象语法树 (AST)，识别列是否允许 NULL、INSERT/UPDATE 时的 NULL 值等。

在解析阶段，主要是标记哪些列或表达式可能包含 NULL。

3.1.3 实现原理

- 在 minioB 数据库中，所有的值由抽象数据类型 value 表示，value 中有一个 bool 类型的成员变量 is_null 用于标记该值是否为 NULL，这使得在存储和处理数据时可以通过判断来统一对待 NULL 值。
- 存储引擎需要在数据块中为每一元组提供表示 NULL 的机制，例如使用位图 (NULL bitmap) 标记 NULL 值，我这里近似使用 bool* 来表示。
- 读取和写入数据时，需要检查并维护该 NULL 位图，以保证 NULL 信息与实际数据一致。
- 对于索引列，存储引擎需要额外处理 NULL 值在索引中的存储和查询。

Value 用于表达数据库中所有类型的值的抽象类 Value 的核心属性如下，在其中添加 is_null_ 作为标记即可统一 NULL 与非 NULL 值的处理。这也是整个 NULL 功能实现的基础。

```
1      class Value
2      {
3          ...
4          private:
5              AttrType attr_type_ = AttrType::UNDEFINED;
6              int      length_     = 0;
7              // 只能显式设置为null类型
8              bool is_null_ = false;
9
10             union Val
11             {
12                 int32_t int_value_;
13                 float   float_value_;
14                 bool    bool_value_;
15                 char    *pointer_value_;
```

```

16         } value_ = {.int_value_ = 0};
17
18         /// 是否申请并占有内存，目前对于 CHARS 类型 own_data_
        为true，其余类型 own_data_ 为false
19         bool own_data_ = false;
20     };

```

Record Record 类用于表示存储引擎中的一条记录，其相较 Tuple 类包含更多的信息。其与 NULL 相关的核心属性和方法如下，在其中添加 is_null_ “位图”用于标记每个字段是否为 NULL，并提供相应的设置和获取方法。

```

1     class Record {
2         ...
3     public:
4         bool is_null(int index) const {
5             if (index < 0 || index >= is_null_.size()) {
6                 LOG_WARN("Record::is_null() out of range, index = %
d !", index);
7                 return false;
8             }
9             return is_null_[index];
10        }
11
12        void set_bitmap(int field_id, int fields_record_size,
bool n) {
13            bool *bitmap = reinterpret_cast<bool *>(data_ +
fields_record_size);
14            bitmap[field_id] = n;
15            LOG_INFO("Set num %d field's bitmap information");
16        }
17
18        bool get_null_information(int field_id, int
fields_record_size)

```

```

19         {
20             const bool *bitmap      = reinterpret_cast<const
bool *>(data_ + fields_record_size);
21             LOG_INFO("Get num %d field's bitmap information");
22             return bitmap[field_id];
23         }
24
25     private:
26         RID      rid_;
27         string key_; // 记录的主键, 用于 lsm-tree 引擎, 需要考
虑重构 Record
28         char  *data_ = nullptr; // Value::data() 获取到的具体
值的指针
29         int    len_   = 0;        // 如果不是 record 自己来管理内
存, 这个字段可能是无效的
30         bool   owner_ = false;    // 表示当前是否由 record 来管理内
存
31         vector<bool> is_null_;
32     };

```

TableScan Operator 表扫描算子需要从底层文件读取记录, 此时需要根据记录的数据指针 `char*` 以及已知的偏移量, 正确获取并还原每个字段的 NULL 状态标记。下面是表扫描算子中 `next` 方法的核心代码片段, 展示了如何在从文件中读取记录的数据后处理 NULL 信息。

```

1     RC TableScanPhysicalOperator::next()
2     {
3         ...
4         while (OB_SUCC(rc = record_scanner_->next(
current_record_))) {
5             ...
6             bool *null_bitmap = reinterpret_cast<bool *>(
current_record_.data() + table_meta.fields_record_size());

```

```

7         for(int i = 0; i < table_meta.bitmap_record_size()
/ sizeof(bool); i++) {
8             if(null_bitmap[i]) {
9                 current_record_.set_is_null(i);
10            }
11        }
12    }
13 }

```

3.1.4 底层文件管理

- 表数据文件（.data）中，NULL 位图通常与实际列数据一起存储。
- 元数据文件（.meta）记录哪些列允许 NULL，以及列在数据文件中的偏移。
- 在删除或更新表数据时，存储引擎必须同步更新 NULL 位图文件或数据块中相关信息。

3.1.5 执行算子与查询处理

- 查询执行算子需要支持 NULL 值比较和逻辑判断，如 IS NULL、IS NOT NULL。
- 在扫描、过滤和投影算子中，NULL 的处理会影响结果集和排序。
- 物理算子在访问存储引擎时，会结合 NULL 位图完成数据过滤。

3.1.6 小结

NULL 的实现虽然对用户透明，但在 MiniOB 中涉及存储引擎位图管理、数据文件与元数据文件的协同维护，以及聚合函数、查询算子等诸多模块对 NULL 值的正确处理。其实现的核心如前所述，但是在接下来介绍的各个功能模块的实现中均可以看到其身影。

3.2 UPDATE 实现流程

1. 语法解析 (yacc_sql.y)：解析表名、赋值列表、条件
 - 位置：src/observer/sql/parser/yacc_sql.y (597-613 行)

- 功能:将 SQL 语句 `UPDATE table SET col1=val1, col2=val2 WHERE condition` 解析为表名、赋值列表和条件表达式
2. 语句创建 (`update_stmt.cpp`): 验证表/字段存在, 绑定表达式
 - 位置: `src/observer/sql/stmt/update_stmt.cpp` (20-154 行)
 - 功能:
 - 验证目标表存在
 - 验证更新字段存在
 - 绑定 SET 表达式 (支持子查询)
 - 绑定 WHERE 条件表达式
 3. 执行器执行 (`update_executor.cpp`):
 - 位置: `src/observer/sql/executor/update_executor.cpp` (35-335 行)
 - 流程:
 - 使用 `RecordScanner` 扫描表记录
 - 应用 WHERE 条件过滤记录
 - 对每条匹配记录, 计算新字段值
 4. 更新记录 (`heap_table_engine.cpp`): 核心函数 `update_record_with_trx()`

```

1 // 更新索引和数据记录的核心函数
2 RC HeapTableEngine::update_record_with_trx(
3     const Record &old_record,
4     const Record &new_record,
5     Trx *trx)
6 {
7     RC rc = RC::SUCCESS;
8
9     // 1. 更新索引: 先删除旧记录, 再插入新记录
10    bool *old_bitmap = reinterpret_cast<bool *>(
11        const_cast<char *>(old_record.data()) +
12        table_meta->fields_record_size());
13
14    bool *new_bitmap = reinterpret_cast<bool *>(
15        const_cast<char *>(new_record.data()) +
16        table_meta->fields_record_size());
17
18    for (Index *index : indexes_) {

```

```

19         const vector<string> &index_fields = index->index_meta().
fields();
20
21         // 检查旧值和新值是否包含 NULL (任一字段为 NULL 则跳过索引
操作)
22         bool old_has_null = false;
23         bool new_has_null = false;
24         vector<const FieldMeta *> fields_meta;
25
26         for (const string &field_name : index_fields) {
27             const FieldMeta *field_meta = table_meta->field(
field_name.c_str());
28             if (field_meta == nullptr) continue;
29
30             fields_meta.push_back(field_meta);
31             int bitmap_index = field_meta->field_id() + table_meta_
->sys_field_num();
32
33             // 检查 NULL 位图
34             bool old_is_null = (bitmap_index >= 0 && bitmap_index <
table_meta->field_num()) ?
35                 old_bitmap[bitmap_index] : false;
36
37             bool new_is_null = (bitmap_index >= 0 && bitmap_index <
table_meta->field_num()) ?
38                 new_bitmap[bitmap_index] : false;
39
40             if (old_is_null) old_has_null = true;
41             if (new_is_null) new_has_null = true;
42         }
43
44         // 删除旧索引条目 (如果非 NULL)
45         if (!old_has_null && !fields_meta.empty()) {
46             rc = index->delete_entry(old_record.data(), &old_record
.rid());
47             if (rc != RC::SUCCESS) {
48                 LOG_WARN("Failed to delete index entry. table=%s,
index=%s, rid=%s, rc=%s",

```

```

49         table_meta->name(), index->index_meta().
name(),
50         old_record.rid().to_string().c_str(),
strrc(rc));
51     return rc;
52 }
53 }
54
55 // 插入新索引条目 (如果非 NULL)
56 if (!new_has_null && !fields_meta.empty()) {
57     // 唯一索引需要先检查键值唯一性
58     if (index->index_meta().is_unique()) {
59         // 构建复合键 (二进制安全)
60         int total_key_length = 0;
61         for (const FieldMeta *field_meta : fields_meta) {
62             total_key_length += field_meta->len();
63         }
64
65         char *key_buffer = new char[total_key_length];
66         int offset = 0;
67
68         // 拼接字段值
69         for (const FieldMeta *field_meta : fields_meta) {
70             const char *field_data = new_record.data() +
field_meta->offset();
71             memcpy(key_buffer + offset, field_data,
field_meta->len());
72             offset += field_meta->len();
73         }
74
75         // 检查键值是否已存在
76         list<RID> existing_rids;
77         rc = index->get_entry(key_buffer, total_key_length,
existing_rids);
78         delete[] key_buffer;
79
80         if (rc == RC::SUCCESS && !existing_rids.empty()) {
81             // 排除自身记录 (索引字段值未变的情况)

```

```

82         bool is_self = false;
83         for (const RID &existing_rid : existing_rids) {
84             if (existing_rid == new_record.rid()) {
85                 is_self = true;
86                 break;
87             }
88         }
89
90         // 如果存在其他记录使用相同键值, 违反唯一约束
91         if (!is_self) {
92             LOG_WARN("Duplicate_key_violates_unique_
constraint. _table=%s, _index=%s",
93                     table_meta->name(), index->
index_meta().name());
94             return RC::RECORD_DUPLICATE_KEY;
95         }
96     }
97 }
98
99     // 插入新索引条目
100     rc = index->insert_entry(new_record.data(), &new_record
.rid());
101     if (rc != RC::SUCCESS) {
102         LOG_WARN("Failed_to_insert_index_entry. _table=%s, _
index=%s, _rid=%s, _rc=%s",
103                 table_meta->name(), index->index_meta().
name(),
104                 new_record.rid().to_string().c_str(),
strrc(rc));
105         return rc;
106     }
107 }
108 }
109
110 // 2. 更新数据记录
111 rc = record_handler->delete_record(&old_record.rid());
112 if (rc != RC::SUCCESS) {
113     LOG_WARN("Failed_to_delete_old_record. _table=%s, _rid=%s, _rc

```

```

114         =%s",
        table_meta_>name(), old_record.rid().to_string().
c_str(), strrc(rc));
115         return rc;
116     }
117
118     rc = record_handler_>insert_record(
119         new_record.data(),
120         new_record.len(),
121         const_cast<RID*>(&new_record.rid()));
122
123     if (rc != RC::SUCCESS) {
124         LOG_WARN("Failed to insert new record. table=%s, rid=%s, rc
=%s",
125             table_meta_>name(), old_record.rid().to_string().
c_str(), strrc(rc));
126         return rc;
127     }
128
129     return RC::SUCCESS;
130 }
131

```

关键设计说明：

(a) **NULL** 值处理：

- 遍历索引字段，检查旧值和新值的 NULL 位图
- 任一字段为 NULL 则跳过索引操作（SQL 标准允许多个 NULL）

(b) 索引更新策略：

- **先删除旧索引条目**：若旧值非 NULL，调用 `index->delete_entry()`
- **再插入新索引条目**：若新值非 NULL，先检查唯一性再插入

(c) 唯一性检查：

- 构建复合键：拼接所有索引字段的二进制数据
- 查询索引获取相同键值的记录列表
- 排除自身记录（允许更新非索引字段）
- 若存在其他记录，返回 `RC::RECORD_DUPLICATE_KEY`

(d) 数据记录更新:

- 先删除旧记录: `record_handler_->delete_record()`
- 再插入新记录: 保持相同 RID 实现原地更新

(e) 更新顺序:

- 先更新索引再更新数据: 保证一致性
- 若先更新数据后更新索引失败, 会导致数据与索引不一致
- 先更新索引可提前发现唯一性冲突

5. 唯一性检查详细流程

(a) 构建复合键:

- 计算总长度: `total_key_length =  $\Sigma$ (field_meta->len())`
- 分配缓冲区: `char* key_buffer = new char[total_key_length]`
- 拼接字段: 按索引字段顺序 `memcpy` 字段数据
- 示例: 索引 (name VARCHAR(20), age INT) $\rightarrow$ 键结构: [20 字节 name] [4 字节 age]

(b) 查询索引检查重复:

- 调用 `index->get_entry(key_buffer, total_key_length, existing_rids)`
- 返回匹配该键的所有 RID 列表

(c) 排除自身记录:

- 遍历返回的 RID 列表
- 检查是否包含当前记录的 RID
- 场景: 更新非索引字段时 (如唯一索引在 id, 更新 name)

(d) 冲突检测:

- 若存在其他记录的 RID, 返回 `RC::RECORD_DUPLICATE_KEY`
- 否则允许更新

6. 完整流程示例

• 表结构:

```
1 CREATE TABLE users (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(20) UNIQUE,  
4     age INT  
5 );
```

- 更新操作: `UPDATE users SET name='Alice' WHERE id=1`
 - (a) 构建新键: 提取 `name='Alice'` 的二进制数据
 - (b) 查询索引: `get_entry("Alice", ...)` 返回 RID 列表
 - (c) 排除自身: 检查返回列表是否包含 `id=1` 的 RID
 - (d) 冲突处理:
 - 若包含: 允许更新 (索引字段值未变)
 - 若不包含且列表非空: 返回 `RECORD_DUPLICATE_KEY`
 - (e) 插入新键: 唯一性检查通过后调用 `insert_entry()`

3.3 Join Table 实现流程

1. 语法解析 (`yacc_sql.y`)
 - 位置: `src/observer/sql/parser/yacc_sql.y` (912-995 行, 998-1030 行)
 - 功能: 解析表引用和 JOIN 条件 (如 `table1 INNER JOIN table2 ON condition`)
 - 输出: 存储到 `TableReferenceSqlNode`, 包含表名、别名和 JOIN 条件
2. 语句创建 (`select_stmt.cpp`)
 - 位置: `src/observer/sql/stmt/select_stmt.cpp` (170-198 行, 530-574 行)
 - 流程:
 - 解析表引用: 验证所有表存在
 - 收集 JOIN 条件: 遍历 `table_references`, 收集到 `all_join_conditions`
 - 创建 JOIN 过滤器: 使用 `FilterStmt::create` 创建全局 JOIN 条件过滤器
3. 逻辑计划生成 (`logical_plan_generator.cpp`)
 - 位置: `src/observer/sql/optimizer/logical_plan_generator.cpp` (111-220 行)
 - 流程:
 - 构建 JOIN 树: 为每个表创建 `TableGetLogicalOperator`
 - 多表时创建 `JoinLogicalOperator`
 - 分配 JOIN 条件: 将条件分配给对应的 `PredicateLogicalOperator`
4. 物理计划生成
 - 位置: `src/observer/sql/optimizer/physical_plan_generator.cpp`

- 算法选择：根据会话参数选择 Hash Join 或 Nested Loop Join

```

1 // 算法选择逻辑
2 if (session->hash_join_on() && can_use_hash_join(join_oper)
3     ) {
4     // 选择 Hash Join
5     auto hash_join_oper = make_unique<
6     HashJoinPhysicalOperator>();
7
8     // 设置连接字段（等值条件）
9     Field left_field, right_field;
10    for (auto &expr : join_conditions) {
11        if (is_equijoin_condition(expr, left_field,
12        right_field)) {
13            hash_join_oper->set_join_fields(left_field,
14            right_field);
15            break;
16        }
17    }
18
19    // 设置过滤表达式（包括非等值条件）
20    hash_join_oper->set_filter_expressions(predicate_oper->
21    expressions());
22
23    oper = std::move(hash_join_oper);
24    LOG_INFO("Using Hash Join for this query");
25 } else {
26     // 选择 Nested Loop Join
27     auto join_physical_oper = make_unique<
28     NestedLoopJoinPhysicalOperator>();
29     oper = std::move(join_physical_oper);
30     LOG_INFO("Using Nested Loop Join for this query");
31 }

```

5. Hash Join 实现

(a) 构建阶段：在右表上构建哈希表

```
1 void HashJoinPhysicalOperator::build_hash_table()
2 {
3     if (right_join_field_.meta() == nullptr) {
4         hash_table_built_ = true;
5         return;
6     }
7
8     while (right_>next() == RC::SUCCESS) {
9         // 获取连接键值
10        Value join_value;
11        get_field_value(right_join_field_, join_value);
12
13        // 跳过 NULL 值
14        if (join_value.is_null()) continue;
15
16        // 物化右表元组并存入哈希表
17        hash_table_[join_value.to_string()].push_back(
18            materialize_tuple(right_tuple_)
19        );
20    }
21 }
22
```

(b) 探测阶段：处理左表行并匹配

```
1 RC HashJoinPhysicalOperator::next()
2 {
3     // 未设置等值键时退化为嵌套循环
4     if (left_join_field_.meta() == nullptr) {
5         return nested_loop_next();
6     }
7
```

```

8      // 正常Hash Join流程
9      if (!current_matches_ || current_match_iter_ ==
current_matches_->end()) {
10         RC rc = probe_hash_table();
11         if (rc != RC::SUCCESS) return rc;
12     }
13
14     // 获取下一个匹配
15     right_tuple_ = *current_match_iter_;
16     ++current_match_iter_;
17
18     // 应用额外过滤条件
19     if (!evaluate_filter_conditions(left_tuple_,
right_tuple_)) {
20         return this->next(); // 跳过不匹配的行
21     }
22
23     // 组合结果
24     joined_tuple_.set_left(left_tuple_);
25     joined_tuple_.set_right(right_tuple_);
26
27     return RC::SUCCESS;
28 }
29

```

(c) 过滤处理:

```

1  bool HashJoinPhysicalOperator::evaluate_filter_conditions
   (Tuple *left_tuple, Tuple *right_tuple)
2  {
3      JoinedTuple joined_tuple(left_tuple, right_tuple);
4
5      for (auto &expr : filter_expressions_) {
6          Value value;

```

```

7         expr->get_value(joined_tuple, value);
8
9         // 任一条件为假则返回 false
10        if (value.is_null() || !value.get_boolean()) {
11            return false;
12        }
13    }
14    return true;
15 }
16

```

6. Nested Loop Join 实现

(a) 主循环:

```

1  RC NestedLoopJoinPhysicalOperator::next()
2  {
3      while (true) {
4          // 如果一轮右表扫描完成, 获取下一左表行
5          if (round_done_) {
6              RC rc = left_next();
7              if (rc != RC::SUCCESS) return rc;
8          }
9
10         // 获取下一右表行
11         RC rc = right_next();
12         if (rc == RC::RECORD_EOF) {
13             round_done_ = true; // 标记右表扫描完成
14             continue;          // 回到循环开始处理下一
左表行
15         }
16
17         // 组合元组并返回
18         joined_tuple_.set_left(left_tuple_);
19         joined_tuple_.set_right(right_tuple_);

```

```

20         return RC::SUCCESS;
21     }
22 }
23

```

(b) 右表控制:

```

1  RC NestedLoopJoinPhysicalOperator::right_next()
2  {
3      if (round_done_) {
4          // 重置右表扫描
5          right_>close();
6          right_>open(trx_);
7          round_done_ = false;
8      }
9
10     RC rc = right_>next();
11     if (rc == RC::RECORD_EOF) {
12         round_done_ = true; // 标记右表扫描完成
13     }
14     return rc;
15 }
16

```

7. JOIN 结果组合

- 使用 JoinedTuple 组合左右表元组
- 文件: src/observer/sql/expr/tuple.h
- 实现原理:

```

1  class JoinedTuple : public Tuple {
2  public:
3      JoinedTuple(Tuple *left, Tuple *right)
4          : left_(left), right_(right) {}
5
6      // 获取字段值 (先左表后右表)
7      RC cell_at(int index, Value &cell) override {

```

```

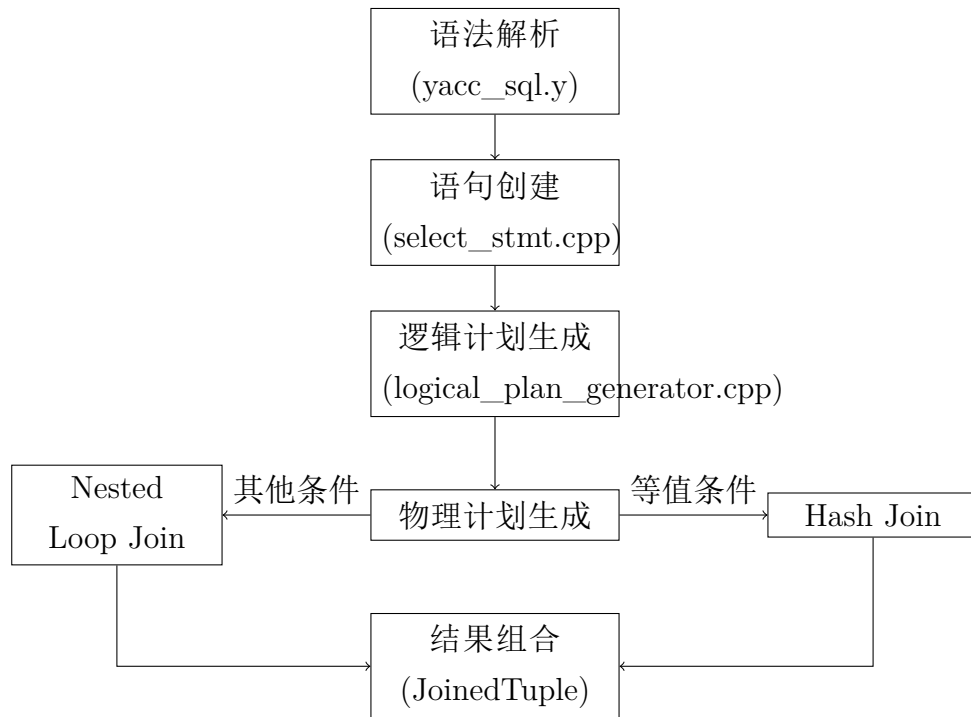
8         if (index < left_>cell_num()) {
9             return left_>cell_at(index, cell);
10        }
11        return right_>cell_at(index - left_>cell_num(),
cell);
12    }
13
14    int cell_num() const override {
15        return left_>cell_num() + right_>cell_num();
16    }
17
18    // 设置左右表元组
19    void set_left(Tuple *left) { left_ = left; }
20    void set_right(Tuple *right) { right_ = right; }
21
22 private:
23     Tuple *left_;
24     Tuple *right_;
25 };
26

```

8. 核心设计总结

- **多表 JOIN 树**: 通过逻辑计划构建 JOIN 树结构
- **算法自适应**:
 - Hash Join: 适用于等值连接, $O(M+N)$ 时间复杂度
 - Nested Loop Join: 通用算法, 支持任意连接条件
- **NULL 处理**: 连接键为 NULL 时不参与匹配
- **统一接口**: 使用 JoinedTuple 透明访问多表字段

JOIN 执行流程示意图



Hash Join vs Nested Loop Join 对比

特性	Hash Join	Nested Loop Join
适用条件	等值连接	任意连接条件
时间复杂度	$O(M + N)$	$O(M \times N)$
内存使用	高 (需建哈希表)	低 (流式处理)
NULL 处理	跳过 NULL 键	处理 NULL 键
实现复杂度	中等	简单

表 1: JOIN 算法特性对比

3.4 复合索引实现流程

1. 语法解析 (yacc_sql.y)

- 位置: src/observer/sql/parser/yacc_sql.y (366-385 行)

- 功能：解析多字段索引（如 `CREATE INDEX idx ON table(id, name)`）
 - 输出：字段名列表 `["id", "name"]`
2. 语句创建 (`create_index_stmt.cpp`)
- 位置：`src/observer/sql/stmt/create_index_stmt.cpp` (24-68 行)
 - 流程：
 - 验证所有字段存在
 - 收集 `FieldMeta` 指针
 - 处理唯一性约束标志
3. 索引元数据 (`index_meta.cpp`)
- 位置：`src/observer/storage/index/index_meta.cpp` (42-65 行)
 - 功能：`IndexMeta` 存储字段名列表 `fields_`，支持单字段和多字段索引
4. 索引创建 (`heap_table_engine.cpp`)
- 位置：`src/observer/storage/table/heap_table_engine.cpp`
 - 流程：
 - 唯一性检查：扫描记录，构建复合键检查重复
 - 创建 B+ 树索引文件（键类型为 `CHARS`，长度为所有字段长度之和）
 - 填充索引：扫描所有记录，插入复合键

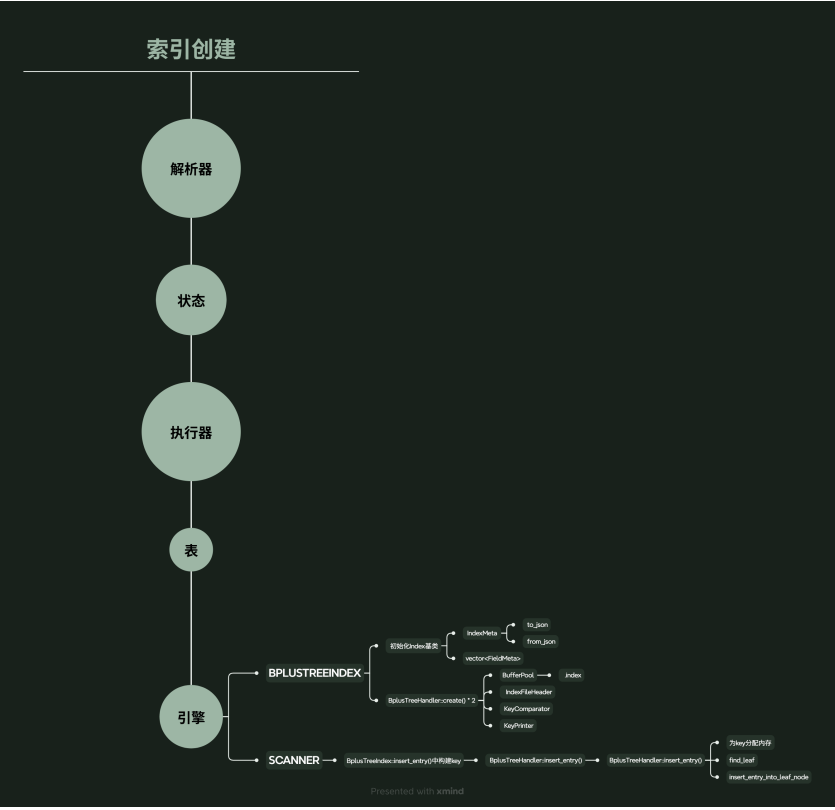


图 9: 索引创建流程图

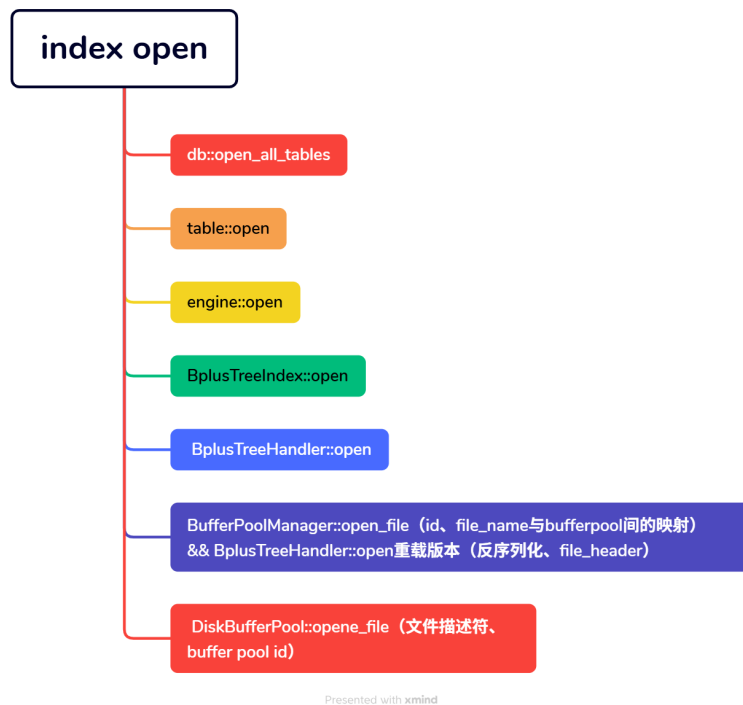


图 10: 打开索引（创建之后使用）流程图

5. 复合键构建

- 位置: BplusTreeIndex::build_composite_key
- 功能: 按索引字段顺序拼接字段的二进制数据

```

1 // 构建复合键的核心代码
2 int total_length = 0;
3 for (const FieldMeta *field_meta : fields_meta_) {
4     total_length += field_meta->len(); // 累加字段长度
5 }
6
7 char *key_buffer = new char[total_length];
8 int offset = 0;
9
10 // 按顺序拼接字段数据
11 for (const FieldMeta *field_meta : fields_meta_) {

```

```

12     const char *field_data = record + field_meta->offset();
13     memcpy(key_buffer + offset, field_data, field_meta->len
14     ());
15     offset += field_meta->len();
16 }

```

- 设计要点:

- 使用字段的固定长度 `field_meta->len()`
- 字符字段不足时用原记录填充保证键长度恒定
- 复合键为” 字段字节段的顺序拼接”，B+ 树视为固定长度字节流

6. 索引扫描的前缀扩展

- 位置: `BplusTreeIndex::open` 中的范围准备逻辑
- 功能: 当查询键长度小于完整复合键时自动补全

```

1 // 左边界补最小值
2 if (left_key && left_len < full_key_length) {
3     for (int j = start_field_idx; j < fields_meta_.size();
4     j++) {
5         const FieldMeta *field_meta = fields_meta_[j];
6         switch (field_meta->type()) {
7             case INTS: fill_value(expanded_left_key +
8             offset, INT_MIN); break;
9             case FLOATS: fill_value(expanded_left_key +
10             offset, -FLT_MAX); break;
11             default: memset(expanded_left_key + offset,
12             0, field_meta->len());
13         }
14         offset += field_meta->len();
15     }
16     final_left_key = expanded_left_key;
17     final_left_len = full_key_length;
18 }

```

```

16 // 右边界补最大值
17 if (right_key && right_len < full_key_length) {
18     for (int j = start_field_idx; j < fields_meta_.size();
19         j++) {
20         const FieldMeta *field_meta = fields_meta_[j];
21         switch (field_meta->type()) {
22             case INTS: fill_value(expanded_right_key +
23 offset, INT_MAX); break;
24             case FLOATS: fill_value(expanded_right_key +
25 offset, FLT_MAX); break;
26             default: memset(expanded_right_key + offset,
27 0xFF, field_meta->len());
28         }
29         offset += field_meta->len();
30     }
31     final_right_key = expanded_right_key;
32     final_right_len = full_key_length;
33 }

```

- 作用：如查询 id=3 时扩展为 [3+min, 3+max]，扫描所有 id=3 的记录

7. 索引操作

- 位置：BplusTreeIndex::insert_entry, delete_entry, get_entry
- 统一接口：

```

1 RC BplusTreeIndex::insert_entry(const char *record, const
  RID *rid) {
2     char key_buf[key_len_];
3     build_composite_key(record, key_buf); // 构建复合键
4     return index_handler_.insert_entry(key_buf, rid); // 调
      用 B+ 树插入
5 }
6
7 RC BplusTreeIndex::delete_entry(const char *record, const

```

```

    RID *rid) {
8      char key_buf[key_len_];
9      build_composite_key(record, key_buf);
10     return index_handler_.delete_entry(key_buf, rid);
11 }
12
13 RC BplusTreeIndex::get_entry(const char *key, int key_len,
    list<RID> &rids) {
14     return index_handler_.get_entry(key, key_len, rids);
15 }
16

```

- 更新场景：
 - 先 delete_entry 旧键
 - 再 insert_entry 新键
 - 唯一索引会先 get_entry 查重

查询条件	扩展后左边界	扩展后右边界
id=3	3 + INT_MIN	3 + INT_MAX
name='Tom'	'Tom' + \0	'Tom' + \xFF
id>3 AND name<'Lee'	3 + min + 'Lee' + min	3 + max + 'Lee' + max

表 2: 复合索引前缀扩展规则

3.5 UNIQUE 索引实现流程

UNIQUE 索引用于保证索引键的唯一性（NULL 值除外），其实现流程涵盖语法解析、元数据存储、创建/插入/更新阶段的唯一性检查，以及 NULL 值处理等环节。

1. 语法解析 (yacc_sql.y)

- 位置：src/observer/sql/parser/yacc_sql.y (356-365 行, 376-385 行)
- 功能：解析 CREATE UNIQUE INDEX 语句时，识别 UNIQUE 关键字并设置索引的 is_unique 标志为 true。

- 语法规则示例：

```

1 CREATE UNIQUE INDEX idx_user_id ON users(id); -- 唯一索引
2 CREATE INDEX idx_user_name ON users(name);      -- 普通索引
3

```

- 解析逻辑：在 Yacc 语法规则中，UNIQUE 作为可选关键字匹配，若存在则将 CreateIndexStmt 的 is_unique 属性设为 true(对应代码中 stmt->set_is_unique(true))

2. 元数据存储 (index_meta.cpp/h)

- 位置：

- 定义：src/observer/storage/index/index_meta.h (47 行, 59 行)
- 实现：src/observer/storage/index/index_meta.cpp (27-40 行)

- 功能：IndexMeta 类通过 is_unique_ 成员变量存储索引的唯一性标志，支持获取和设置操作。

- 代码片段：

```

1 // index_meta.h (定义)
2 class IndexMeta {
3 private:
4     bool is_unique_; // 唯一索引标志 (true表示唯一)
5 public:
6     bool is_unique() const { return is_unique_; } // 获取唯一性状态
7     void set_is_unique(bool is_unique) { is_unique_ = is_unique; } // 设置唯一性状态
8 };
9
10 // index_meta.cpp (实现)
11 IndexMeta::IndexMeta(const char* name, const std::vector<
12     const FieldMeta*>& fields, bool is_unique)
13     : name_(name), fields_(fields), is_unique_(is_unique)
14     {}
15

```

3. 创建索引时的唯一性检查

- 逻辑：创建唯一索引前，通过 RecordScanner 全表遍历，构建索引键并使用哈希表 key_to_rid 查重。若发现重复键，立即返回 RC::RECORD_DUPLICATE_KEY，终止索引创建。
- NULL 处理：若索引字段包含 NULL（通过记录的 NULL 位图判断），则跳过该记录的唯一性检查（允许多个 NULL）。
- 代码片段（heap_table_engine.cpp）：

```

1 // 创建唯一索引时的全表唯一性检查
2 std::unordered_map<std::string, RID> key_to_rid; // 哈希
   表：键→RID
3 RecordScanner check_scanner(table_meta_, record_handler_);
4 RC rc = check_scanner.open();
5 if (OB_FAIL(rc)) return rc;
6
7 while (OB_SUCC(rc = check_scanner.next(record))) {
8     // 1. 检查索引字段是否为 NULL（通过 bitmap 判断）
9     bool is_field_null = false;
10    const char* bitmap = record.data() + table_meta_>
fields_record_size(); // NULL 位图位置
11    for (const FieldMeta* field : index_meta.fields()) {
12        int field_id = field->field_id();
13        if (bitmap[field_id / 8] & (1 << (field_id % 8))) {
14            // 位图位为 1 表示 NULL
15            is_field_null = true;
16            break;
17        }
18    }
19    if (is_field_null) continue; // NULL 值跳过检查
20
21    // 2. 构建索引键（复合键为字段字节拼接）
22    std::string key_str;
23    for (const FieldMeta* field : index_meta.fields()) {
24        const char* field_data = record.data() + field->
offset();

```

```

24         key_str.append(field_data, field->len()); // 拼接字
    段字节
25     }
26
27     // 3. 查重：若键已存在，返回重复错误
28     if (key_to_rid.find(key_str) != key_to_rid.end()) {
29         check_scanner.close();
30         return RC::RECORD_DUPLICATE_KEY;
31     }
32     key_to_rid[key_str] = record.rid(); // 键→RID映射
33 }
34
35 check_scanner.close();
36

```

4. 插入记录时的唯一性检查

- 逻辑:插入记录前,对每个唯一索引,构建索引键并查询索引(index->get_entry)。若索引中存在相同键,返回 RC::RECORD_DUPLICATE_KEY,拒绝插入。
- **NULL 处理**: 若索引键包含 NULL,跳过检查(允许多个 NULL)。
- 代码片段(heap_table_engine.cpp):

```

1 // 插入记录时的唯一索引检查
2 for (Index* index : indexes_) {
3     if (!index->index_meta().is_unique()) continue; // 非唯
    一索引跳过
4
5     // 1. 检查索引字段是否为NULL (通过 bitmap 判断)
6     bool is_field_null = false;
7     const char* bitmap = record.data() + table_meta->
    fields_record_size();
8     for (const FieldMeta* field : index->index_meta().
    fields()) {
9         int field_id = field->field_id();
10        if (bitmap[field_id / 8] & (1 << (field_id % 8))) {

```

```

11         is_field_null = true;
12         break;
13     }
14 }
15 if (is_field_null) continue;
16
17 // 2. 构建复合键
18 int total_len = 0;
19 for (const FieldMeta* field : index->index_meta().
20 fields()) {
21     total_len += field->len();
22 }
23 char* key_buffer = new char[total_len];
24 int offset = 0;
25 for (const FieldMeta* field : index->index_meta().
26 fields()) {
27     const char* field_data = record.data() + field->
28 offset();
29     memcpy(key_buffer + offset, field_data, field->len
30 ());
31     offset += field->len();
32 }
33
34 // 3. 查询索引：是否存在相同键
35 std::list<RID> existing_rids;
36 RC rc = index->get_entry(key_buffer, total_len,
37 existing_rids);
38 delete[] key_buffer;
39 if (rc == RC::SUCCESS && !existing_rids.empty()) {
40     return RC::RECORD_DUPLICATE_KEY; // 存在重复键，拒
41     绝插入
42 }
43 }

```

5. 更新记录时的唯一性检查

- 逻辑：更新记录时，需删除旧键并插入新键。插入新键前，对每个唯一索引进行检查：
 - (a) 构建新键；
 - (b) 查询索引是否存在相同键；
 - (c) 若存在，排除当前记录自身 (`existing_rid == new_record.rid()`)，若仍有其他记录，则返回重复错误。
- 代码片段 (`heap_table_engine.cpp`):

```

1 // 更新记录时的唯一索引检查 (插入新键前)
2 for (Index* index : indexes_) {
3     if (!index->index_meta().is_unique()) continue;
4
5     // 1. 检查新记录的索引字段是否为 NULL (通过 bitmap 判
6     断)
7     bool new_has_null = false;
8     const char* new_bitmap = new_record.data() +
9     table_meta->fields_record_size();
10    for (const FieldMeta* field : index->index_meta().
11    fields()) {
12        int field_id = field->field_id();
13        if (new_bitmap[field_id / 8] & (1 << (field_id % 8)
14        )) {
15            new_has_null = true;
16            break;
17        }
18    }
19    if (new_has_null) continue;
20
21    // 2. 构建新复合键
22    int total_key_length = 0;
23    for (const FieldMeta* field : index->index_meta().

```

```

20     fields()) {
21         total_key_length += field->len();
22     }
23     char* key_buffer = new char[total_key_length];
24     int offset = 0;
25     for (const FieldMeta* field : index->index_meta().
fields()) {
26         const char* field_data = new_record.data() + field
->offset();
27         memcpy(key_buffer + offset, field_data, field->len
());
28         offset += field->len();
29     }
30     // 3. 查询索引是否存在相同键
31     std::list<RID> existing_rids;
32     RC rc = index->get_entry(key_buffer, total_key_length,
existing_rids);
33     delete[] key_buffer;
34     if (rc == RC::SUCCESS && !existing_rids.empty()) {
35         // 排除当前记录自身（索引字段未变更的情况）
36         bool is_self = false;
37         for (const RID& rid : existing_rids) {
38             if (rid == new_record.rid()) {
39                 is_self = true;
40                 break;
41             }
42         }
43         if (!is_self) {
44             return RC::RECORD_DUPLICATE_KEY; // 存在其他记
录的重复键
45         }
46     }

```

47

}

48

- 排除自身的必要性：更新非索引字段时（如唯一索引在 `id`，更新 `name`），索引键未变，查询会返回当前记录自身，需排除以避免误判冲突。

6. NULL 值处理

- 逻辑：遵循 SQL 标准（ISO/IEC 9075），NULL 视为“未知”，不与任何值相等（包括自身）。因此，唯一索引允许多个 NULL 值（NULL 不会被视为重复）。
- 实现位置：src/observer/storage/table/heap_table_engine.cpp (309-320 行, 523 行)

核心设计要点

- 三阶段一致性保障：
 1. 创建时：全表扫描检查历史数据重复；
 2. 插入时：实时查询索引检查新键冲突；
 3. 更新时：排除自身后检查新键冲突。
- SQL 标准兼容：严格支持 NULL 值语义，允许多个 NULL 共存。
- 性能优化：
 - 创建时全表扫描仅执行一次；
 - 插入/更新时通过索引查询（ $O(\log n)$ ）而非全表扫描；
 - 复合键使用二进制拼接，哈希表/索引查询高效。
- 冲突处理：通过返回 `RC::RECORD_DUPLICATE_KEY` 明确拒绝重复键，保证数据一致性。

3.6 子查询实现流程

子查询是 SQL 中嵌套在其他查询内部的查询语句，支持在 WHERE 条件、UPDATE SET 子句、比较表达式和 IN/NOT IN 等场景中使用。以下是子查询在 MiniOB 中的实现流程：

1. 语法解析 (yacc_sql.y)

- 位置：src/observer/sql/parser/yacc_sql.y (601-651 行, 1151-1211 行)

- 功能：解析形如 (SELECT ...) 的子查询结构，创建 `subquery_stmt` 节点，并用 `SubqueryExpr` 包装解析树。
- 支持场景：
 - WHERE 条件中的子查询：SELECT * FROM t1 WHERE id IN (SELECT id FROM t2)
 - UPDATE SET 表达式：UPDATE t1 SET col = (SELECT MAX(val) FROM t2)
 - 比较表达式：SELECT * FROM t1 WHERE col > (SELECT AVG(col) FROM t2)
 - IN/NOT IN 子句

2. 表达式绑定 (`select_stmt.cpp`)

- 位置：src/observer/sql/stmt/select_stmt.cpp (452-500 行)
- 流程：
 - (a) 为 `SubqueryExpr` 创建 `SelectStmt` 对象
 - (b) 传递表上下文：子查询可访问外层查询的表（支持相关子查询）
 - (c) 合法性检查：确保标量子查询只返回单个属性
 - (d) 绑定子查询内部的表达式
- 代码片段：

```

1 // 创建子查询 SelectStmt
2 if (expr->type() == ExprType::SUB_QUERY) {
3     SubqueryExpr *subquery_expr = static_cast<SubqueryExpr
4     *>(expr.get());
5     RC rc = SelectStmt::create(db,
6                                *subquery_expr->sub_query_sn
7                                (),
8                                stmt,
9                                subquery_expr->stmt_ref());
10    if (rc != RC::SUCCESS) return rc;
11
12    // 检查标量子查询只返回单列
13    if (subquery_expr->stmt()->projects().size() != 1) {
14        return RC::SUBQUERY_TOO_MANY_COLUMNS;
15    }
16 }

```

```

13     }
14 }
15

```

3. 逻辑计划生成

- 位置: LogicalPlanGenerator
- 功能: 为子查询生成逻辑算子树, 并存储到 SubqueryExpr 中
- 接口设计:

```

1 class SubqueryExpr {
2 private:
3     std::unique_ptr<LogicalOperator> logical_operator_;
4     std::unique_ptr<PhysicalOperator> physical_operator_;
5 public:
6     void set_logical_operator(std::unique_ptr<
7 LogicalOperator> logical_operator) {
8         logical_operator_ = std::move(logical_operator);
9     }
10    void set_physical_operator(std::unique_ptr<
11 PhysicalOperator> physical_operator) {
12        physical_operator_ = std::move(physical_operator);
13    }
14 };
15

```

- 流程: 表达式绑定后, 调用 LogicalPlanGenerator 生成子查询的逻辑算子树, 并通过 set_logical_operator() 存储。

4. 物理计划生成

- **TableGet** 场景:

```

1 // 处理比较表达式中的子查询
2 if (comparison_expr->left()->type() == ExprType::SUB_QUERY)
3 {
4     auto sub_query_expr = static_cast<SubqueryExpr*>(
5         comparison_expr->left().get());
6 }
7

```

```

4     unique_ptr<PhysicalOperator> subquery_phy_oper;
5     RC rc = create(*sub_query_expr->logical_operator(),
6                   subquery_phy_oper,
7                   session);
8     sub_query_expr->set_physical_operator(std::move(
9     subquery_phy_oper));
10  }
11  // 右子树同理

```

- **Predicate 场景:**

```

1  // 为过滤条件中的子查询生成物理算子
2  if (comparison_expr->left()->type() == ExprType::SUB_QUERY)
3  {
4      auto sub_query_expr = static_cast<SubqueryExpr*>(expr.
5      get());
6      if (session) sub_query_expr->set_trx(session->
7      current_trx());
8      unique_ptr<PhysicalOperator> subquery_phy_oper;
9      RC rc = create(*sub_query_expr->logical_operator(),
10                    subquery_phy_oper,
11                    session);
12      sub_query_expr->set_physical_operator(std::move(
13      subquery_phy_oper));
14  }

```

- 说明：物理算子生成后存储到 SubqueryExpr 中，供执行阶段使用。

5. 执行阶段

- 流程:

- (a) 打开物理算子（传入外层元组指针）
- (b) 获取首行首列值
- (c) 检查行数：多行报错，无行返回 NULL
- (d) 关闭物理算子

- 代码片段:

```
1 RC SubqueryExpr::get_value(Tuple *outer_tuple, Value &value
  ) {
2     // 1. 打开物理算子 (传入外层元组)
3     RC rc = open_physical_operator(outer_tuple);
4     if (rc != RC::SUCCESS) return rc;
5
6     // 2. 获取首行
7     rc = physical_operator_>next();
8     if (rc == RC::RECORD_EOF) { // 无结果返回 NULL
9         value.set_null();
10        close_physical_operator();
11        return RC::SUCCESS;
12    }
13
14    // 3. 读取第一列
15    Tuple *sub_t = physical_operator_>current_tuple();
16    rc = sub_t->cell_at(0, value);
17
18    // 4. 检查是否多行
19    RC next_rc = physical_operator_>next();
20    if (next_rc == RC::SUCCESS) {
21        close_physical_operator();
22        return RC::SUBQUERY_MULTIPLE_ROWS; // 多行报错
23    }
24
25    // 5. 关闭算子
26    close_physical_operator();
27    return rc;
28 }
29
```

6. 相关子查询支持

- 机制：通过 `set_outer_tuple()` 传递外层元组指针
- 代码实现：

```

1 // 打开子查询物理算子时传递外层元组
2 RC open_physical_operator(Tuple *outer_tuple) {
3     physical_operator_>set_outer_tuple(outer_tuple);
4     return physical_operator_>open(trx_);
5 }
6

```

7. 绑定阶段合法性检查

- **SELECT** 场景：

```

1 if (condition.left_expr->type() == ExprType::SUB_QUERY) {
2     SelectStmt::create(db, subquery_sn->selection, stmt);
3     check_sub_select_legal(subquery_expr->stmt()); // 检查
      单列
4 }
5

```

- **UPDATE** 场景：

```

1 if (value_expr->type() == ExprType::SUB_QUERY) {
2     SelectStmt::create(db, subquery_sn->selection, nullptr)
      ; // 无外层表
3     Stmt::check_sub_select_legal(subquery_expr->stmt());
4 }
5

```

- 规则：
 - 标量子查询必须返回单列
 - UPDATE SET 中的子查询不能引用外层表

8. UPDATE 中的子查询

- 位置：src/observer/sql/stmt/update_stmt.cpp (55-83 行, 105-127 行)
- 流程：
 - (a) 解析 SET 子句中的子查询表达式

- (b) 创建 `SelectStmt`
- (c) 检查子查询是否只返回单列
- (d) 绑定表达式

设计要点总结

- 分层处理：语法解析 → 表达式绑定 → 逻辑计划 → 物理计划 → 执行
- 表达式封装：通过 `SubqueryExpr` 统一管理子查询的逻辑和物理算子
- 相关子查询：通过 `outer_tuple` 机制实现外层数据传递
- 执行优化：惰性执行（需要时才打开算子）
- 安全机制：
 - 标量子查询强制单列检查
 - `UPDATE SET` 子查询禁止引用外层表
 - 多行结果自动报错

3.7 ALIAS（别名）实现流程

别名机制用于为表或字段提供临时名称，提升查询可读性和灵活性。MiniOB 支持表别名和字段别名，并处理了多层嵌套查询中的别名遮蔽问题。

1. 语法解析 (`yacc_sql.y`)

- 位置：`src/observer/sql/parser/yacc_sql.y` (772-812 行, 900 行)
- 功能：
 - 表别名：解析 `table AS alias` 或 `table alias` 结构，存储到 `RelationSqlNode.alias`
 - 字段别名：解析 `expression AS alias` 或 `expression alias` 结构，调用 `expression->set_alias()`

2. 表别名处理

- 收集与映射：
 - 构建 `table_alias_map` 映射：别名 → 真实表名
 - 检查重复：同一查询层内不允许别名指向不同表
- 代码实现：

```

1 // 构建别名映射并检查重复
2 std::unordered_map<std::string, std::string>
   table_alias_map;

```

```

3 for (const auto &a : select_sql.ALIASES) {
4     if (visible_tables.count(a.name) == 0) continue; // 跳
    过外层表
5     if (table_alias_map.find(a.alias) != table_alias_map.
    end() &&
6         table_alias_map[a.alias] != a.name) {
7         return RC::INVALID_ARGUMENT; // 别名冲突
8     }
9     table_alias_map[a.alias] = a.name;
10 }
11
12 // 补充 JOIN表的别名
13 for (const auto &tr : select_sql.table_references) {
14     if (!tr.alias.empty() && visible_tables.count(tr.
    table_name)) {
15         table_alias_map[tr.alias] = tr.table_name;
16     }
17 }
18

```

- 注册与传递:

```

1 // 注册到绑定上下文
2 for (const auto &kv : table_alias_map) {
3     binder_context.add_table_alias(kv.first.c_str(), table)
4     ;
5 }
6
7 // 向子查询传递别名映射
8 if (name2alias) name2alias->insert({real_table, alias});
9 if (alias2name) alias2name->insert({alias, real_table});

```

- 遮蔽规则: 本层别名可遮蔽外层同名别名

3. 字段别名处理

- 收集：构建 `field_alias_map`（别名 $\rightarrow$ 原始字段名）
- 限制：星号 (*) 不允许带别名
- 代码实现：

```

1  std::unordered_map<std::string, std::string>
    field_alias_map;
2  for (auto *expr : select_sql.expressions) {
3      if (expr->type() == ExprType::STAR) {
4          if (static_cast<StarExpr*>(expr)->has_alias()) {
5              return RC::INVALID_ARGUMENT; // 星号禁止别名
6          }
7      } else if (expr->has_alias()) {
8          field_alias_map[expr->alias()] = expr->name();
9      }
10 }
11

```

- 用途：ORDER BY/GROUP BY 子句可引用字段别名
4. 别名解析 (select_stmt.cpp)
- 位置：src/observer/sql/stmt/select_stmt.cpp (343-380 行)
 - 流程：
 - (a) 表别名解析：将 `UnboundFieldExpr` 中的别名映射到真实表名
 - (b) 字段别名解析：在 ORDER BY 等子句中解析字段别名
 - (c) 默认绑定：单表查询时未指定表名的字段自动绑定
5. 别名绑定 (expression_binder.cpp)
- 位置：src/observer/sql/parser/expression_binder.cpp (23-39 行)
 - 机制：
 - 表别名查找：`BinderContext::find_table(alias)`
 - 字段别名查找：通过 `field_alias_map` 映射
 - 查找顺序：
 - (a) 本层表别名
 - (b) 本层表名
 - (c) 外层别名（支持相关子查询）

6. 别名传递与遮蔽

- 传递机制：通过 `name2alias/alias2name` 映射向子查询传递别名
- 遮蔽规则：子查询可访问外层别名，但本层别名优先
- 代码实现：

```
1 // 绑定字段表达式时的别名处理
2 if (field_expr->table_name()) {
3     auto iter = table_alias_map.find(table_name);
4     if (iter != table_alias_map.end()) { // 本层别名
5         field_expr->set_real_table(iter->second);
6     } else if (visible_tables.count(table_name)) { // 本层
        表名
7         // 直接使用
8     } else { // 尝试外层
9         Table *outer_table = outer_context->find_table(
        table_name);
10        if (outer_table) field_expr->set_real_table(
        outer_table->name());
11    }
12 }
13
```

7. 别名输出

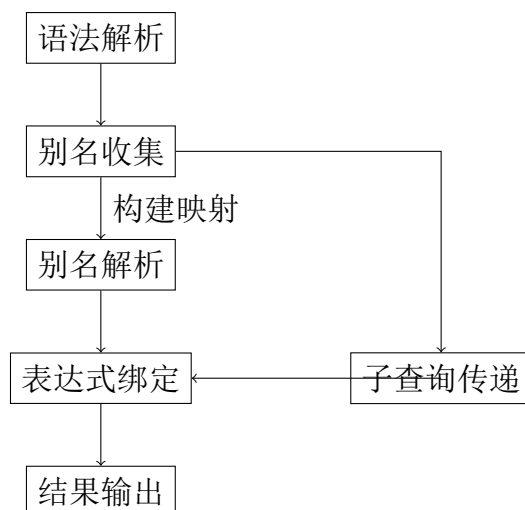
- 位置：src/observer/sql/operator/project_physical_operator.cpp (82-84 行)
- 机制：Project 算子使用字段别名构建输出 Schema
- 代码实现：

```
1 // 构建输出 Schema
2 for (auto &expr : expressions_) {
3     TupleCellSpec spec(expr->name().c_str(), expr->alias().
        c_str());
4     tuple_schema_.add_cell_spec(spec);
5 }
6
```

- 效果：查询结果列名优先使用字段别名

原始列名	输出列名
salary	annual_income
name	employee_name

别名处理流程图



设计要点总结

- 分层管理：语法解析 → 别名收集 → 解析绑定 → 结果输出
- 作用域控制：
 - 表别名：本层优先，支持遮蔽外层
 - 字段别名：仅当前查询层有效
- 错误处理：
 - 别名冲突检测（同层重复）
 - 星号别名禁止
- 传递机制：通过 `name2alias/alias2name` 实现别名继承
- 输出优化：结果集列名使用字段别名提升可读性

别名遮蔽示例

层级	别名	真实表	可见性
外层	emp	employees	子查询可见
子查询	emp	managers	遮蔽外层同名

3.8 Drop Table 功能实现

3.8.1 概述

Drop Table 是数据库中用于删除表及其相关数据的操作。在 MiniOB 中，该功能涉及 SQL 语法解析、执行器以及底层存储文件的删除等，这里重点关注对于数据文件、索引文件和元数据文件的处理。

文件删除通常通过数据库、表以及存储引擎三层所提供的接口完成，功能核心是删除如下三种文件，它们包含了表的全部信息。

- 数据文件（**.data**）：存储表的所有元组数据。
- 索引文件（**.idx**）：存储表的索引信息，必须同步删除以避免产生孤立索引。（可能存在不止一个）
- 元数据文件（**.meta**）：记录表的 schema 等元数据信息。

3.8.2 语法与词法解析

在 MiniOB 中，用户的 DROP TABLE 命令首先经过词法解析和语法解析：

- 词法解析（**Lexer**）：将输入的 SQL 字符串分解为标记（tokens），识别关键字 DROP、TABLE 以及表名。
- 语法解析（**Parser**）：将 token 序列转换为抽象语法树（AST），生成对应的 DropTableStmt 对象，用于将信息传递到执行器中进行后续处理的规划。

3.8.3 执行器（Executor）

- MiniOB 中的执行器负责将语法树中的 DROP TABLE 映射到下层的具体操作。
- DropTableExecutor 会检查表是否存在，验证相关约束，并调用数据库层的删除操作。

3.8.4 三层接口的实现

Database 层 如下所示是数据库层的表删除接口，其核心代码如下：

- 找到数据库实例中维护的对应表实例，清除数据库实例为其维护的所有资源。
- 获取表对应的存储引擎，并调用表实例的下层接口 drop 完成文件的删除和下层资源的清理。

```
1 RC Db::drop_table(const char *table_name)
2 {
3     RC rc = RC::SUCCESS;
4     Table *table_to_drop = find_table(table_name);
5
6     ...
7
8     // drop函数待定义 --> 清理资源
9     rc = table_to_drop->drop(
10         this,
11         table_id,
12         table_file_path.c_str(),
13         table_name,
14         path_.c_str(),
15         get_storage_engine()
16     );
17
18     ...
19
20     // 清除打开表的记录
21     string table_name_string(table_name);
22     opened_tables_.erase(table_name_string);
23
24     ...
25 }
```

Table 层 如下所示是表层的删除接口，其核心代码如下：

- 根据获取到的路径，将表的数据文件和元文件进行删除。
- 继续调用下层接口，清理该表对应的存储引擎和缓冲区资源。

```

1 RC Table::drop(Db *db, int32_t table_id, const char *meta_path,
    const char *table_name, const char *base_dir, StorageEngine
    storage_engine)
2 {
3     ...
4
5     // 查找表的元数据，确保该表存在
6     std::string name1 = table_meta_.name();
7     std::string name2 = table_name;
8
9     // 关闭并清理表的存储引擎
10    if (engine_) {
11        // 自定义close，关闭索引等 (--> 内部调用了buffer pool的
        close_file --> 进一步调用manager的close_file)
12        RC rc = engine_->close();
13        if (rc != RC::SUCCESS) {
14            LOG_WARN("Failed to close table engine: %s", table_meta_.
                name());
15            return rc;
16        }
17        engine_.reset();
18    }
19
20    string data_file = table_data_file(base_dir, table_name);
21    // db为传入的this
22    BufferPoolManager &bpm = db->buffer_pool_manager();
23    // close buffer pool 中的文件并且进行删除
24    RC rc = bpm.drop_file(data_file.c_str());
25
26    if (remove(meta_path) != 0) {
27        LOG_ERROR("Failed to remove table meta file: %s", meta_path

```

```

    );
28     return RC::IOERR_DELETE;
29 }
30
31 ...
32 }

```

Storage engine 层 如下所示是存储引擎层的表删除接口，负责清除索引和记录处理器，释放相关资源。

```

1 RC HeapTableEngine::close()
2 {
3     // 关闭索引
4     indexes_.clear();
5
6     // 关闭记录处理器
7     if (record_handler_ != nullptr) {
8         // 已经完整实现
9         record_handler_->close();
10        delete record_handler_;
11        record_handler_ = nullptr;
12    }
13
14    ...
15 }

```

3.8.5 小结

Drop Table 操作在 MiniOB 中虽然从用户视角简单，但在内核中涉及语法解析、执行器调度以及多文件删除。通过逐层传递信息并对数据文件、索引文件和元数据文件等进行处理，可以保证操作的便捷性、正确性和数据库的一致性。

3.9 ORDER BY 功能实现

3.9.1 概述

ORDER BY 是 SQL 中用于对查询结果进行排序的操作，在查询中至关重要。在 MiniOB 中，ORDER BY 的实现涉及语法解析、逻辑算子构建、物理算子执行以及记录排序等，这里重点关注算子及其下层操作的实现。如下所示是我在开发过程中绘制的简单思维导图，仅供参考：

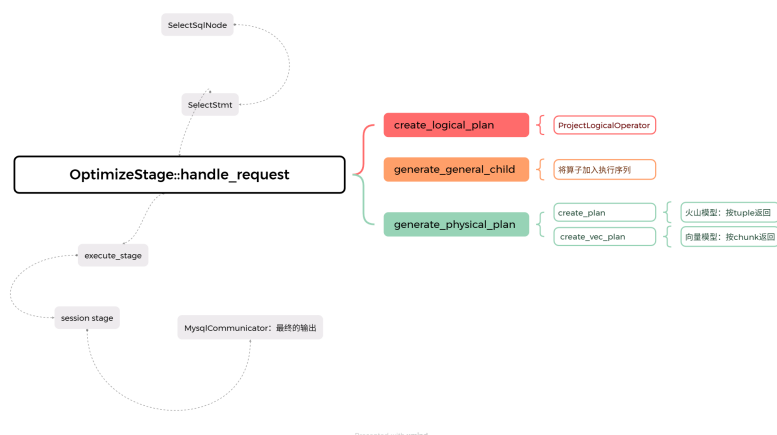


图 11: order by 实现流程图

3.9.2 语法与词法解析

- 词法解析 (**Lexer**): 将 SQL 查询字符串分解，然后识别 order by，并递归识别其后的排序字段和升降序标识。
- 语法解析 (**Parser**): 生成使用表达式存储了 order by 字段与升降序信息的查询语句对象 (**SelectStmt**)。

3.9.3 Order By 逻辑算子 (Order By Logical Operator)

- MiniOB 会在逻辑计划阶段生成相应的逻辑算子，如 **LogicalSort**。
- 逻辑算子描述排序操作的语义，而不关心具体的执行细节。
- 可以与其他逻辑算子（如扫描、投影、过滤等）组合形成完整的查询计划。

3.9.4 Order By 物理算子 (Order By Physical Operator)

物理算子功能概述

- 物理算子将逻辑计划映射为具体的操作，每个物理算子最终均会在执行算子树时，获取前面算子的结果，然后对其执行自身的逻辑。
- MiniOB 的元组获取采用从底层数据文件流式读取，并向上传递的实现方式（扫描算子）。order by 的物理算子会将流在内存处进行截断，直到流中所有的目标元组全部获取出来，再统一排序并向上层算子或客户端传递。

物理算子核心代码 根据前面的功能概述，ORDER BY 物理算子的核心执行流程可以概括为三个阶段：数据收集、内存排序以及结果输出，其关键代码结构如下所示。

在物理算子的执行中，open 用于截断数据流，通过子算子提前将所有符合条件的元组获取出来，然后使用 sort_buffer 进行内存排序。

```
1   RC OrderByPhysicalOperator::open(Trx *trx)
2   {
3       PhysicalOperator *child = children_.front().get();
4       child->open(trx);
5
6       while (child->next() == RC::SUCCESS) {
7           Tuple *tuple = child->current_tuple();
8           ValueListTuple *copy = new ValueListTuple();
9           ValueListTuple::make(*tuple, *copy);
10          tuples_buffer.emplace_back(copy);
11      }
12
13      return sort_buffer();
14  }
```

sort_buffer 支持多字段、升降序的方式，其排序依据主要是物理算子从上层获取的排序表达式（包含字段、升降序信息）。

```
1   RC OrderByPhysicalOperator::sort_buffer()
2   {
```

```

3      std::sort(tuples_buffer.begin(), tuples_buffer.end(),
4                [this](ValueListTuple *a, ValueListTuple *b) {
5
6          for (const auto &expr : order_by_expressions_) {
7              Value va, vb;
8              a->nonstrict_get_value(expr->field_name(), va);
9              b->nonstrict_get_value(expr->field_name(), vb);
10
11              int cmp = va.compare(vb);
12              if (cmp != 0) {
13                  return (expr->order() == DESC) ? (cmp > 0) : (
14                      cmp < 0);
15              }
16              return true;
17          }
18      );
19      return RC::SUCCESS;
20  }

```

next 是所有算子流式获取下一个元组的统一接口。由于 order by 物理算子已经在 open 中将所有目标元组全部取出，因此这里只需要进行下标的递增即可。

```

1      RC OrderByPhysicalOperator::next()
2      {
3          index++;
4          return RC::SUCCESS;
5      }

```

currnet_tuple 是所有算子将元组读取出来，用于向上传递的统一接口。

```

1      Tuple *OrderByPhysicalOperator::current_tuple()
2      {
3          if (index >= tuples_buffer.size()) {
4              return nullptr;
5          }

```

```

6         return tuples_buffer[index];
7     }

```

算子执行流程分析 从执行流程上看，ORDER BY 物理算子的行为具有明显的阶段性。

首先，ORDER BY 算子并不直接访问底层数据文件。其下层算子（如表扫描算子）通过存储引擎读取数据页，并逐条构造逻辑元组（`Tuple`）返回给 ORDER BY 算子。ORDER BY 与存储引擎之间的交互边界由于算子明确隔离。

其次，在算子打开阶段，ORDER BY 会完整遍历子算子的输出，并将每个元组拷贝为 `ValueListTuple` 缓存在 `tuples_buffer` 中。这一设计表明 MiniOB 当前的 ORDER BY 实现采用纯内存排序，不支持外部排序，也不具备流式输出能力。

排序阶段通过 `std::sort` 完成，其比较逻辑严格按照 ORDER BY 子句中字段的声明顺序执行。对于每个排序字段，算子从元组中提取对应的 `Value`，并通过 `Value::compare()` 完成类型无关的比较。若当前字段值相等，则继续比较下一个字段，直至得出确定的顺序关系或遍历完所有排序字段。

在排序完成后，ORDER BY 算子进入结果输出阶段。此时 `next()` 方法仅维护一个递增索引，`current_tuple()` 直接返回已排序缓冲区中的对应元素，不再涉及任何排序或存储操作。

内存管理与资源释放 由于 ORDER BY 在执行过程中显式分配了 `ValueListTuple` 对象，算子在关闭阶段需要逐一释放缓存中的元组，并清空内部缓冲区。该过程由 `close()` 方法统一完成，体现了 MiniOB 中物理算子对内存资源生命周期的显式管理方式。

```

RC OrderByPhysicalOperator::close()
{
    LOG_INFO("Execute order by physical operator's close method");
    RC rc = RC::SUCCESS;
    // 清理所有分配的内存
    for (auto *ptr : tuples_buffer) {
        if (ptr != nullptr) {

```


- 唯一索引测试-唯一约束通过:

```

miniob >
miniob > INSERT INTO multi_index VALUES (2, 1, 11.2, 'a', '2021-01-02', 1, 1);
SUCCESS

```

- 子查询测试:

```

miniob > select * from seq_3;
id | col1 | feat1
---+-----+-----
1 | 4 | 11.2
2 | 2 | 12
3 | 3 | 11.5

miniob > select * from seq_2;
id | col2 | feat2
---+-----+-----
1 | 2 | 11
2 | 7 | 10.5
3 | 3 | 12.6

miniob > select * from seq_1 where id in (select seq_2.id from seq_2);
id | col1 | feat1
---+-----+-----
1 | 4 | 11.2
2 | 2 | 12

```

- 别名测试:

```

miniob > select * from seq_1;
id | col1 | feat1
---+-----+-----
1 | 4 | 11.2
2 | 2 | 12
3 | 3 | 11.5

miniob > select * from seq_2;
id | col2 | feat2
---+-----+-----
1 | 2 | 11
2 | 7 | 10.5
3 | 3 | 12.6

miniob > select * from seq_1 where id in (select seq_2.id from seq_2);
id | col1 | feat1
---+-----+-----
1 | 4 | 11.2
2 | 2 | 12

miniob > select id as i from seq_1 where id in (select seq_2.id from seq_2);
i
--
1
2

miniob > select id as i from seq_1 as s1 where id in (select seq_2.id as i from seq_2);
i
--
1
2

```

5 总结

本设计实现了基于 MiniOB 的数据库核心功能扩展，涵盖：

- UPDATE 操作的原子性与索引一致性
- JOIN 查询的优化执行（Hash Join/Nested Loop Join）
- 复合索引的前缀扫描与唯一约束
- 子查询的标量执行与相关子查询
- 别名的多层解析与遮蔽机制